PROJECT -JAVA(OOTS)

Topic: -UNIT CONVERTOR

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class UnitConverter {

    private JFrame frame;
    private JTextField inputField;
    private JTextField resultField;
    private JComboBox<String> categoryComboBox, fromComboBox,
toComboBox;

    public static void main(String[] args) {
        EventQueue.invokeLater(() -> {
            try {
                UnitConverter window = new UnitConverter();
                window.frame.setVisible(true);
            } catch (Exception e) {
                e.printStackTrace();
            }
        });
    }

    public UnitConverter() {
        initialize();
    }
```

```
  private void initialize() {
    frame = new JFrame();
    frame.setTitle("Unit Converter");
    frame.setBounds(100, 100, 500, 400);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.getContentPane().setLayout(new BorderLayout());

    // Colorful panel
    JPanel panel = new JPanel();
    panel.setBackground(new Color(255, 228, 225)); // Soft pink
background
    frame.getContentPane().add(panel, BorderLayout.CENTER);
    panel.setLayout(new GridLayout(5, 2, 10, 10));

    // Label for category
    JLabel categoryLabel = new JLabel("Select Category:");
    categoryLabel.setForeground(new Color(0, 102, 204)); // Blue text
    panel.add(categoryLabel);

    String[] categories = {"Length", "Mass", "Numerical System",
"Temperature", "GST"};
    categoryComboBox = new JComboBox<>(categories);
    categoryComboBox.setBackground(new Color(255, 255, 255)); // White
background
    categoryComboBox.setForeground(new Color(0, 102, 204)); // Blue text
    panel.add(categoryComboBox);

    // Label for "From"
    JLabel fromLabel = new JLabel("From:");
    fromLabel.setForeground(new Color(0, 102, 204)); // Blue text
    panel.add(fromLabel);
```

```
    fromComboBox = new JComboBox<>();
    fromComboBox.setBackground(new Color(255, 255, 255)); // White
background
    fromComboBox.setForeground(new Color(0, 102, 204)); // Blue text
    panel.add(fromComboBox);

    // Label for "To"
    JLabel toLabel = new JLabel("To:");
    toLabel.setForeground(new Color(0, 102, 204)); // Blue text
    panel.add(toLabel);

    toComboBox = new JComboBox<>();
    toComboBox.setBackground(new Color(255, 255, 255)); // White
background
    toComboBox.setForeground(new Color(0, 102, 204)); // Blue text
    panel.add(toComboBox);

    // Label for input field
    JLabel inputLabel = new JLabel("Enter value:");
    inputLabel.setForeground(new Color(0, 102, 204)); // Blue text
    panel.add(inputLabel);

    inputField = new JTextField();
    inputField.setBackground(new Color(255, 255, 255)); // White
background
    inputField.setForeground(new Color(0, 102, 204)); // Blue text
    panel.add(inputField);
    inputField.setColumns(10);

    // Label for result field
    JLabel resultLabel = new JLabel("Result:");
    resultLabel.setForeground(new Color(0, 102, 204)); // Blue text
```

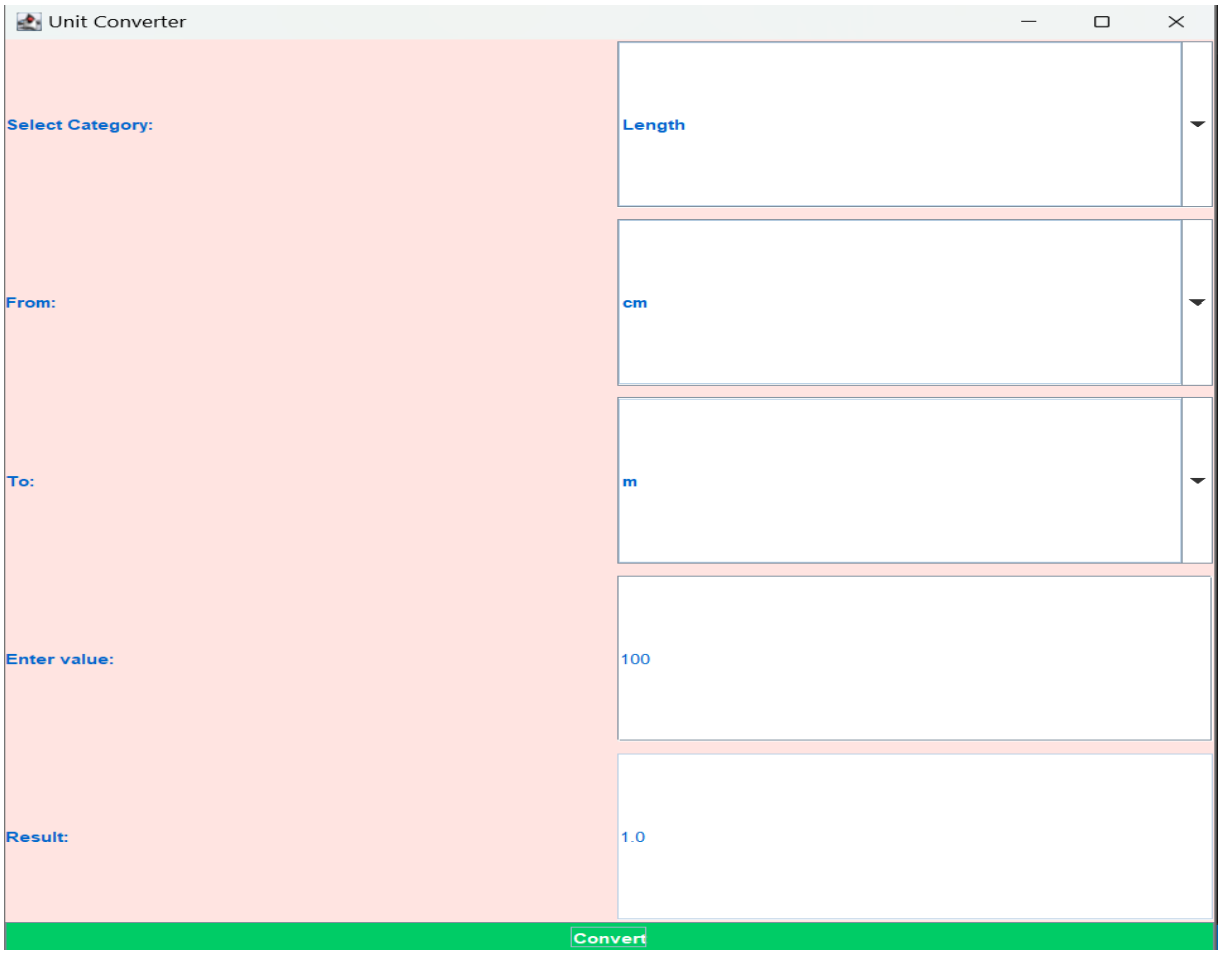
```
        panel.add(resultLabel);

        resultField = new JTextField();
        resultField.setEditable(false);
        resultField.setBackground(new Color(255, 255, 255)); // White
background
        resultField.setForeground(new Color(0, 102, 204)); // Blue text
        panel.add(resultField);
        resultField.setColumns(10);

        // Colorful convert button
        JButton convertButton = new JButton("Convert");
        convertButton.setBackground(new Color(0, 204, 102)); // Green
background
        convertButton.setForeground(Color.WHITE); // White text
        convertButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                convert();
            }
        });
        frame.getContentPane().add(convertButton, BorderLayout.SOUTH);

        // Initial population of ComboBoxes based on category
        categoryComboBox.addActionListener(e -> updateComboBoxes());
        updateComboBoxes();
    }

    private void updateComboBoxes() {
        String category = (String) categoryComboBox.getSelectedItem();

        // Clear previous options
        fromComboBox.removeAllItems();
```

```
toComboBox.removeAllItems();

switch (category) {
  case "Length":
    fromComboBox.addItem("m");
    fromComboBox.addItem("dm");
    fromComboBox.addItem("cm");
    fromComboBox.addItem("mm");
    fromComboBox.addItem("nm");
    fromComboBox.addItem("pm");
    toComboBox.addItem("m");
    toComboBox.addItem("dm");
    toComboBox.addItem("cm");
    toComboBox.addItem("mm");
    toComboBox.addItem("nm");
    toComboBox.addItem("pm");
    break;

  case "Mass":
    fromComboBox.addItem("kg");
    fromComboBox.addItem("g");
    fromComboBox.addItem("mg");
    toComboBox.addItem("kg");
    toComboBox.addItem("g");
    toComboBox.addItem("mg");
    break;

  case "Numerical System":
    fromComboBox.addItem("bin");
    fromComboBox.addItem("oct");
    fromComboBox.addItem("dec");
    fromComboBox.addItem("hex");
```

```
            toComboBox.addItem("bin");
            toComboBox.addItem("oct");
            toComboBox.addItem("dec");
            toComboBox.addItem("hex");
            break;

        case "Temperature":
            fromComboBox.addItem("°C");
            fromComboBox.addItem("°F");
            fromComboBox.addItem("K");
            fromComboBox.addItem("°R");
            fromComboBox.addItem("°Re");
            toComboBox.addItem("°C");
            toComboBox.addItem("°F");
            toComboBox.addItem("K");
            toComboBox.addItem("°R");
            toComboBox.addItem("°Re");
            break;

        case "GST":
            fromComboBox.addItem("Original Price");
            toComboBox.addItem("GST (3%)");
            toComboBox.addItem("GST (5%)");
            toComboBox.addItem("GST (12%)");
            toComboBox.addItem("GST (18%)");
            toComboBox.addItem("GST (28%)");
            break;
    }
}

private void convert() {
    String category = (String) categoryComboBox.getSelectedItem();
```

```
        String fromUnit = (String) fromComboBox.getSelectedItem();
        String toUnit = (String) toComboBox.getSelectedItem();

        try {
            double value = Double.parseDouble(inputField.getText());
            double result = 0.0;

            switch (category) {
                case "Length":
                    result = convertLength(fromUnit, toUnit, value);
                    break;
                case "Mass":
                    result = convertMass(fromUnit, toUnit, value);
                    break;
                case "Numerical System":
                    result = convertNumericalSystem(fromUnit, toUnit, value);
                    break;
                case "Temperature":
                    result = convertTemperature(fromUnit, toUnit, value);
                    break;
                case "GST":
                    result = calculateGST(fromUnit, toUnit, value);
                    break;
            }

            resultField.setText(String.valueOf(result));

        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(frame, "Invalid input! Please enter
a valid number.");
        }
    }
```

```
    private double convertLength(String fromUnit, String toUnit, double
value) {
        double meterValue = 0;

        switch (fromUnit) {
            case "m": meterValue = value; break;
            case "dm": meterValue = value / 10; break;
            case "cm": meterValue = value / 100; break;
            case "mm": meterValue = value / 1000; break;
            case "nm": meterValue = value / 1e9; break;
            case "pm": meterValue = value / 1e12; break;
        }

        switch (toUnit) {
            case "m": return meterValue;
            case "dm": return meterValue * 10;
            case "cm": return meterValue * 100;
            case "mm": return meterValue * 1000;
            case "nm": return meterValue * 1e9;
            case "pm": return meterValue * 1e12;
            default: return 0;
        }
    }

    private double convertMass(String fromUnit, String toUnit, double value) {
        double kgValue = 0;

        switch (fromUnit) {
            case "kg": kgValue = value; break;
            case "g": kgValue = value / 1000; break;
            case "mg": kgValue = value / 1e6; break;
```

```
    }

    switch (toUnit) {
      case "kg": return kgValue;
      case "g": return kgValue * 1000;
      case "mg": return kgValue * 1e6;
      default: return 0;
    }
  }

  private double convertNumericalSystem(String fromUnit, String toUnit,
double value) {
    switch (fromUnit) {
      case "bin":
        if (toUnit.equals("oct")) return
Integer.parseInt(Integer.toBinaryString((int) value), 8);
        if (toUnit.equals("dec")) return
Integer.parseInt(Integer.toBinaryString((int) value), 10);
        if (toUnit.equals("hex")) return
Integer.parseInt(Integer.toBinaryString((int) value), 16);
        break;
      // Add more conversions as needed
    }
    return value;
  }

  private double convertTemperature(String fromUnit, String toUnit, double
value) {
    // Implement temperature conversion logic as shown in the previous
response
    return value; // Placeholder
  }
```

```
  private double calculateGST(String fromUnit, String toUnit, double value)
{
    if (fromUnit.equals("Original Price")) {
      double gstRate = 0;
      switch (toUnit) {
        case "GST (3%)": gstRate = 3; break;
        case "GST (5%)": gstRate = 5; break;
        case "GST (12%)": gstRate = 12; break;
        case "GST (18%)": gstRate = 18; break;
        case "GST (28%)": gstRate = 28; break;
      }
      return value + (value * gstRate / 100);
    }
    return 0;
  }
}
```